

Quantum Resource Management Interface (QRMI) and Slurm Integration

Munetaka Ohtani
Senior Software Engineer
IBM Quantum

Hiroshi Horii
Senior Manager of QCSC and
Head of IBM Quantum Japan,
Senior Technical Staff Member
IBM Quantum



Highlights

We introduce a unified resource management solution that delivers the **Quantum Resource Management Interface (QRMI)** to abstract vendor-specific quantum hardware and uses the **SPANK plugins** framework to control hybrid classical–quantum workflows in Slurm.

- Native QPU integration with Workflow Manager - **Slurm**
- Unified job submission for quantum + classical tasks.
- Extensible and adaptable architecture for other QPU vendors.
- Improved usability, scheduling, and resource control.
- Extensible to cloud-native environments.

- **Fully open-source** stack with easy installation
- Partnered with **10+ organizations.**
- Deployed to **4 HPC data centers** (STFC, RPI and others) and started evaluations
- Enabled **partner's experiments for scientific breakthroughs.**
- Continuously enhancing the stack based on partner feedback

Slurm User Experience

HPC user: use slurm quantum resource

backend is selected in the *slurm resource definition* used for jobs, e.g. *ibm_fez*

resource defined in slurm parameters (configuration)

```
#SBATCH --time=100
#SBATCH --output=<LOGS_PATH>
#SBATCH --gres=qpu:1
#SBATCH --qpu=ibm_fez
#SBATCH --... # other options

srun ...
```

slurm resource is picked up by *application source code* and used for transpilation and execution:

use previously defined resource

```
from qiskit import QuantumCircuit
from qrmi_primitives import QRMIService
# using an IBM QRMI flavor:
from qrmi_primitives.ibm import SamplerV2

# define circuit
circuit = ...

# inject credentials needed for accessing the service at this point
load_dotenv()

# instantiate QRMI service and get quantum resource
service = QRMIService()

resources = service.resources()
qrmi = resources[0]

# Generate transpiler target and transpile
target = get_target(qrmi)
pm = generate_preset_pass_manager(
    optimization_level=1,
    target=target,
)
isa_circuit = pm.run(circuit)

# run the circuit
sampler = SamplerV2(qrmi, options={})
```

IBM Quantum

HPC admin: define access to physical quantum resources

slurm quantum plugin configuration defines which resources are made available for use (e.g. *ibm_fez*)

```
{
  "resources": [
    {
      "name": "ibm_fez",
      "type": "qiskit-runtime-service",
      "environment": {
        "QRMI_IBM_QRS_ENDPOINT": "https://quantum.cloud.ibm.com/api/v1",
        "QRMI_IBM_QRS_IAM_ENDPOINT": "https://iam.cloud.ibm.com",
        "QRMI_IBM_QRS_IAM_APIKEY": "...",
        "QRMI_IBM_QRS_SERVICE_CRN": "..."
      }
    },
    {
      "name": "FRESNEL",
      "type": "pasqal-cloud",
      "environment": {
        "QRMI_PASQAL_CLOUD_PROJECT_ID": "...",
        "QRMI_PASQAL_CLOUD_AUTH_TOKEN": "..."
      }
    }
  ]
}
```

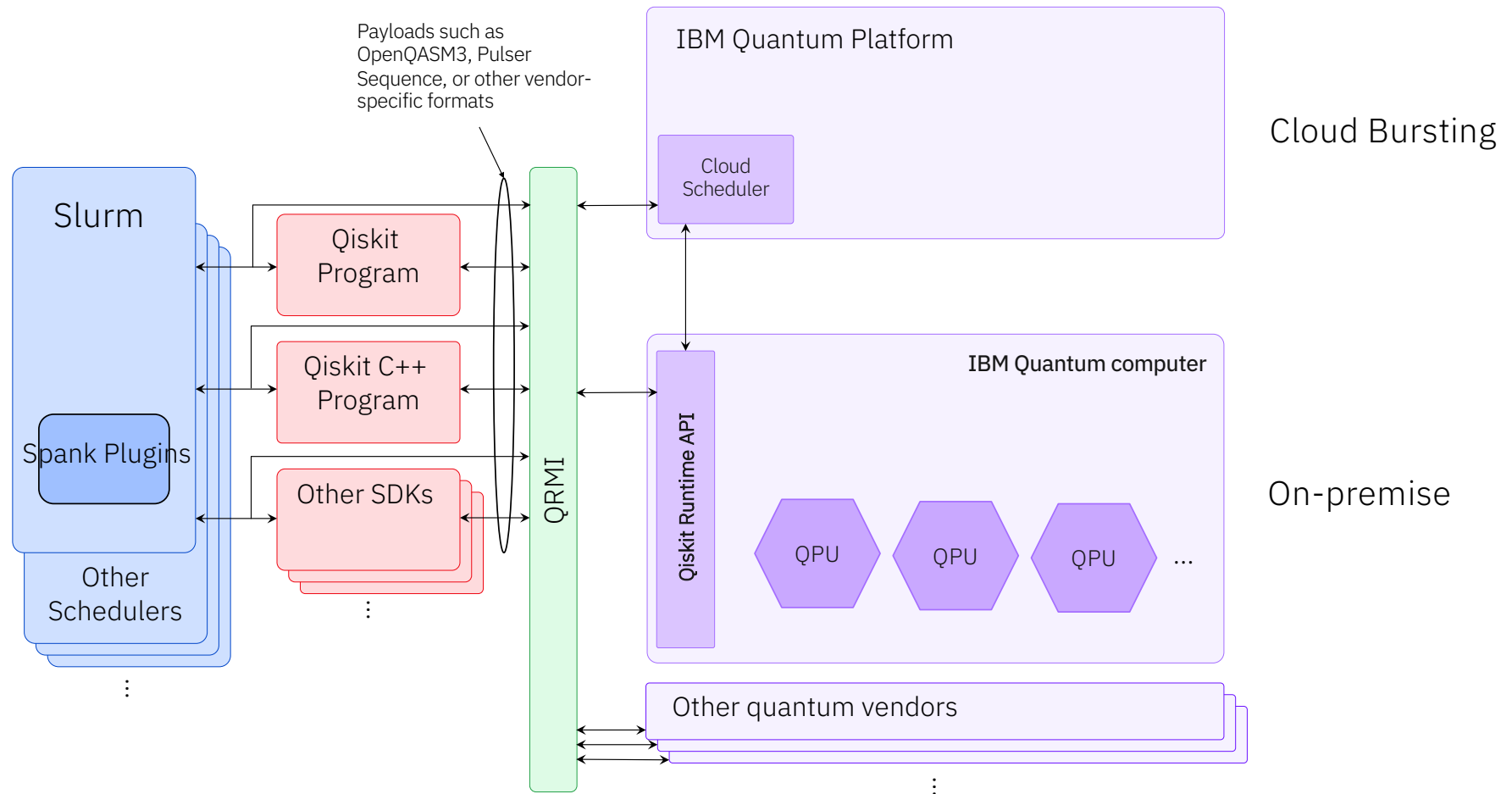
In *slurm.conf*, qpu generic resources can be made available to nodes:

```
...
GresTypes=qpu,name
NodeName=node[1-5000] Gres=qpu,name:ibm_fez
...
```



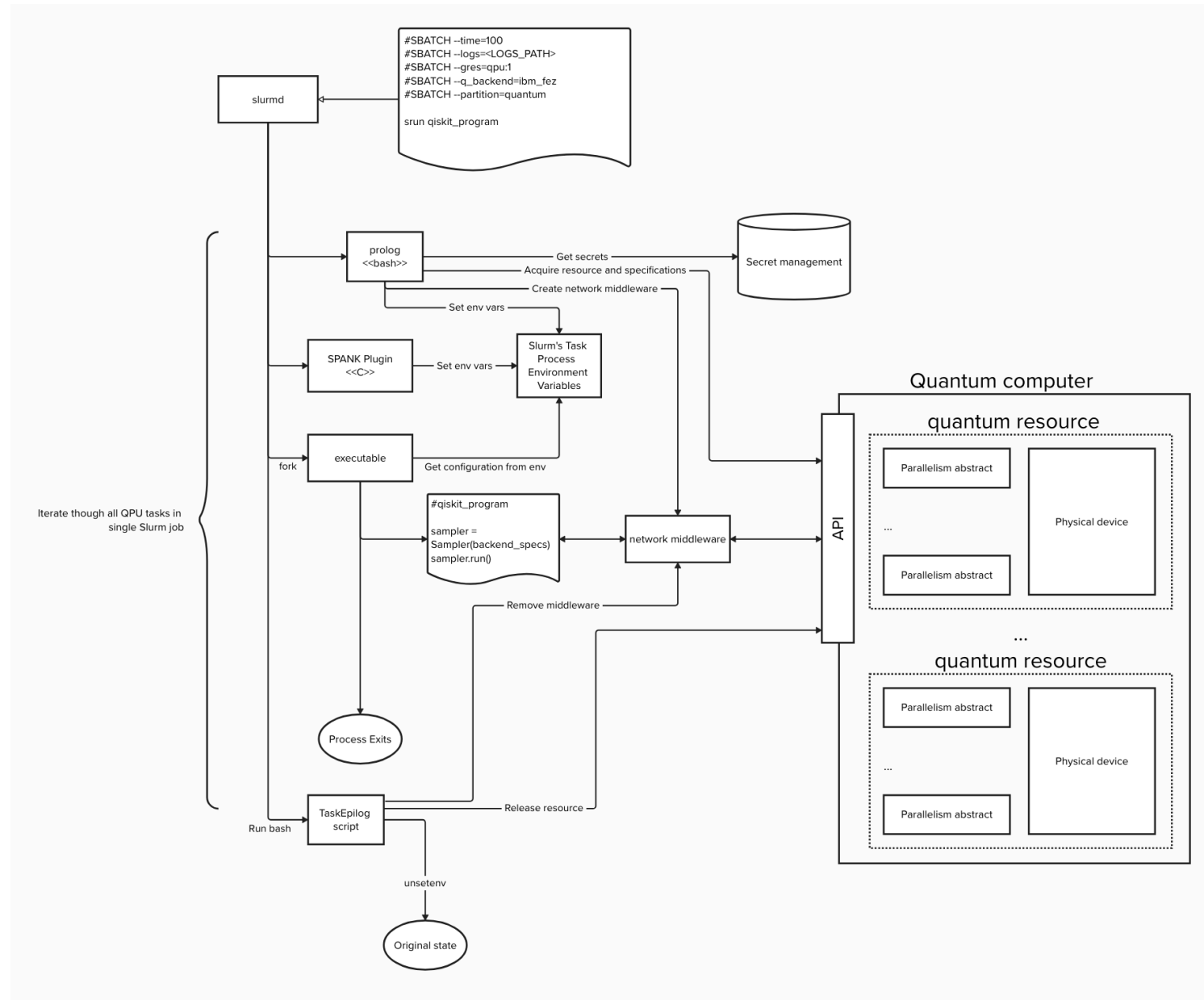
See ux documentation on github for more details
<https://github.com/qiskit-community/spank-plugins/blob/main/docs/ux.md>

QRMI / Slurm



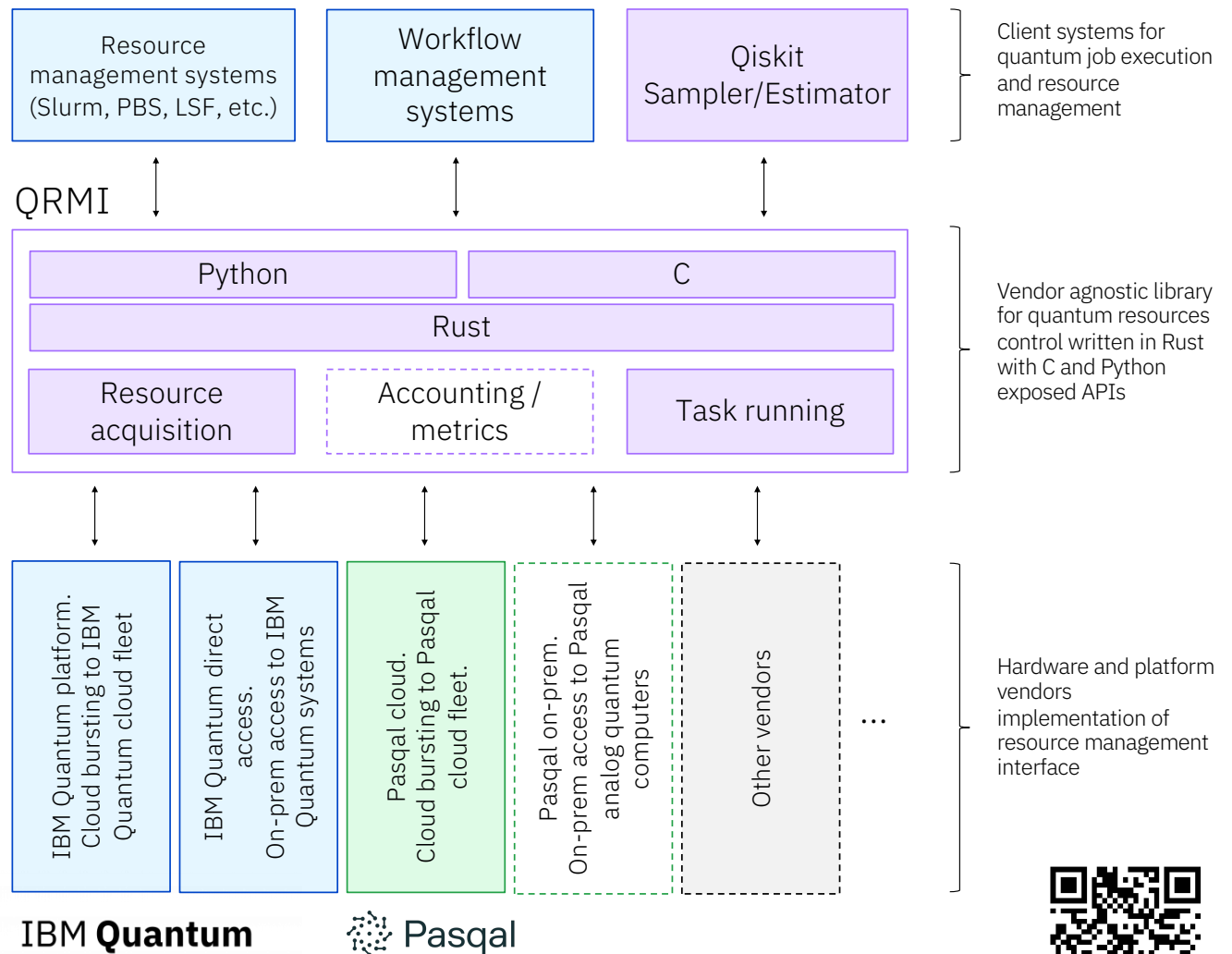
General Architecture of Plugin <job execution>

- SPANK plugin based
- Flow
 - Prolog
 - Acquire resource
 - Task Init
 - Setup env
 - Epilog
 - Release resource
- Postprocess classically quantum results.



Quantum Resource Management Interface (QRMI)

- Thin and vendor agnostic layer to access, control and monitor underlying on-prem or cloud Quantum systems
- Exposes notion of quantum resources, which might be qubit, entire physical system, virtual QPU, etc.
- 3 sets of APIs available for QRMI:
 - Resource acquisition: availability, acquire, release.
 - Task running: execution on quantum payload on target hardware
 - [2026 1H] Accounting/metrics: system utilization, etc.
- Provides loose coupling between clients and systems



QuantumResource

- Defines interfaces to quantum resources. Each Quantum vendor must implement this Rust crate.
- <https://github.com/qiskit-community/qarmi/blob/main/src/lib.rs>



also used by Slurm plugin

Function	Descriptions
is_accessible	Returns true if device is accessible, otherwise false.
acquire	Acquires quantum resource and returns acquisition token if succeeded. If no one owns the lock, it acquires the lock and returns immediately. If another owns the lock, block until we are able to acquire lock.
release	Releases quantum resource
task_start	Start a task and returns an identifier of this task if succeeded.
task_stop	Stops the task specified by `task_id`. This function is called if the user cancels the job or if the time limit for job execution is exceeded. The implementation must cancel the task if it is still running.
task_status	Returns the current status of the task specified by `task_id`.
task_result	Returns the results of the task.
task_logs	Returns the log messages of the task.
target	Returns a Target for the specified device. Vendor specific serialized data. This might contain the constraints(instructions, properties and timing information etc.) of a particular device to allow compilers to compile an input circuit to something that works and is optimized for a device.
metadata	Returns other specific to system or device data

Your QRMI is configured via environment variables.

Common variables. Reserved

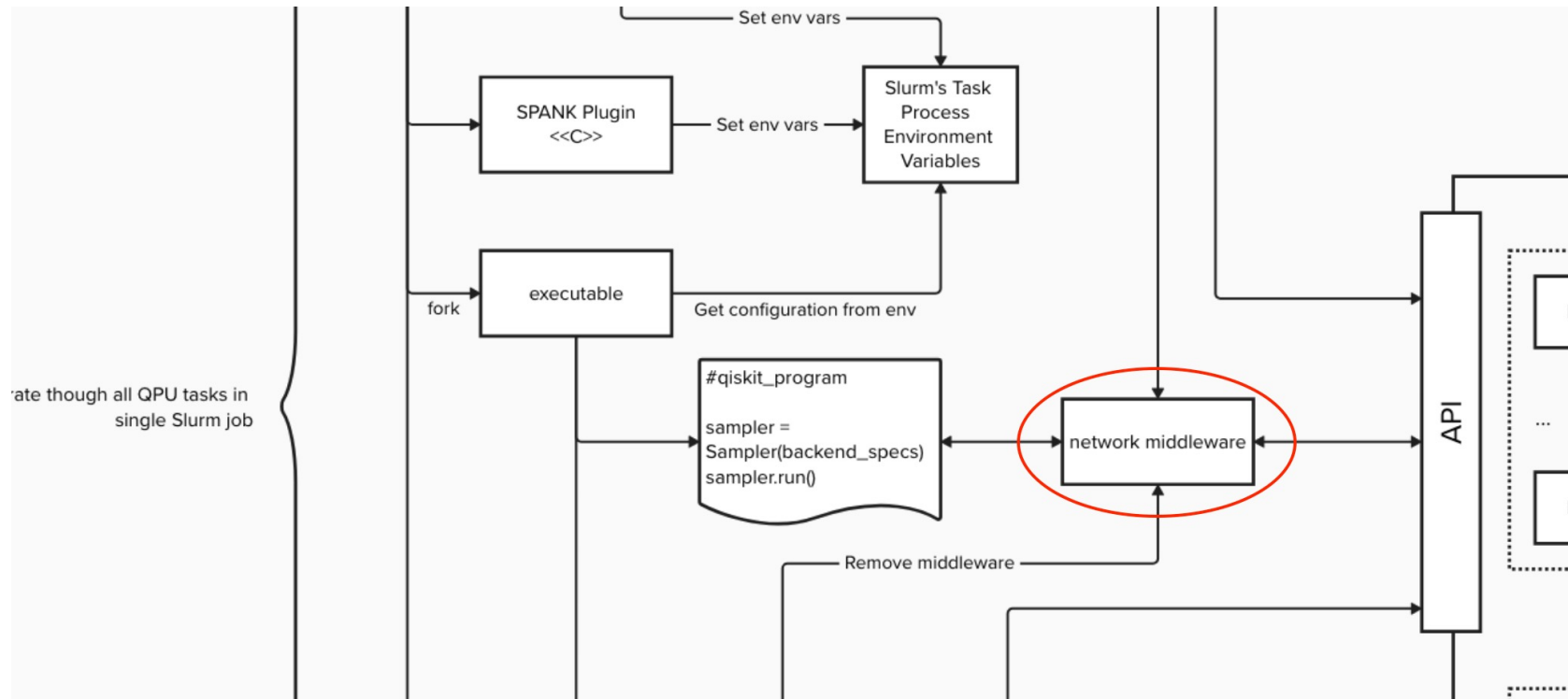
- SLURM_JOB_QPU_RESOURCES
- SLURM_JOB_QPU_TYPES
- QRMI_PLUGIN_ERROR
- {resource_name}_QRMI_JOB_ACQUISITION_TOKEN
- {resource_name}_QRMI_JOB_TIMEOUT_SECONDS

Vendor-defined variables

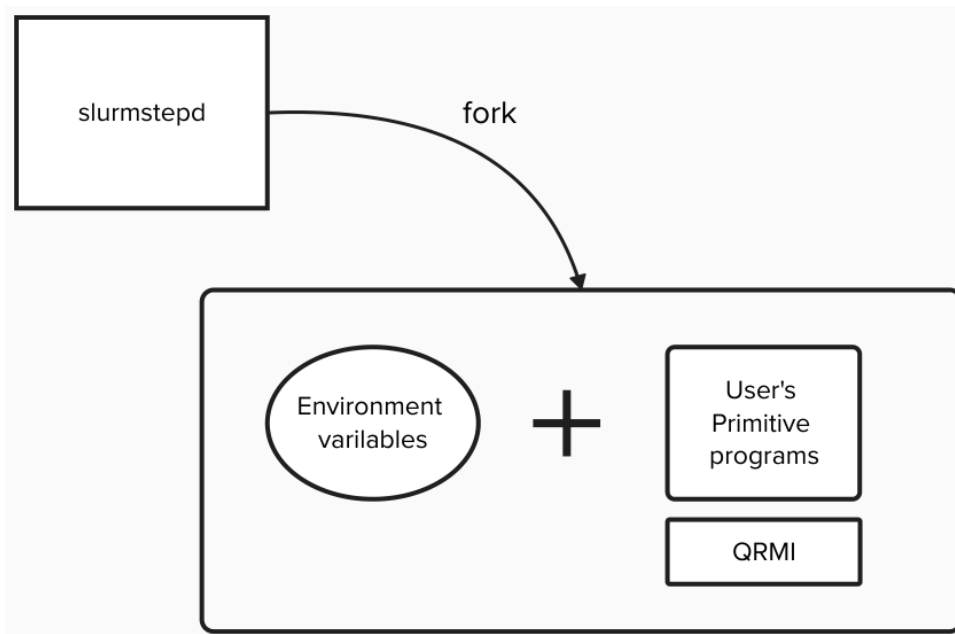
{resource_name}_QRMI_{vendor_name}_{resource_type}_variable_name

- Example:
 - ibm_kingston_QRMI_IBM_QRS_IAM_ENDPOINT

Network middleware



Security issues

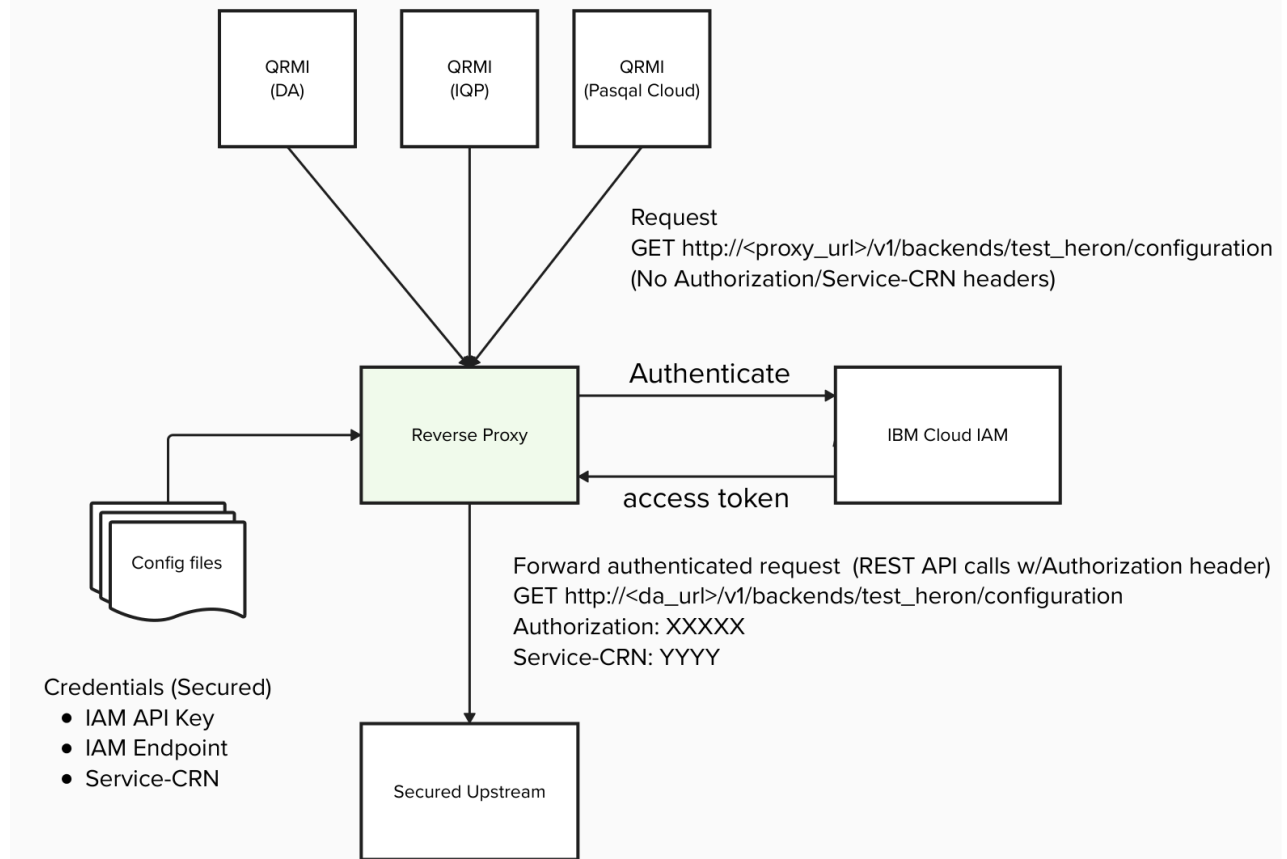


- Configuration parameters including credentials are specified via environment variables
- User can know the cluster wide credentials by retrieving all environment variables from code.

Network middleware to resolve the security issues

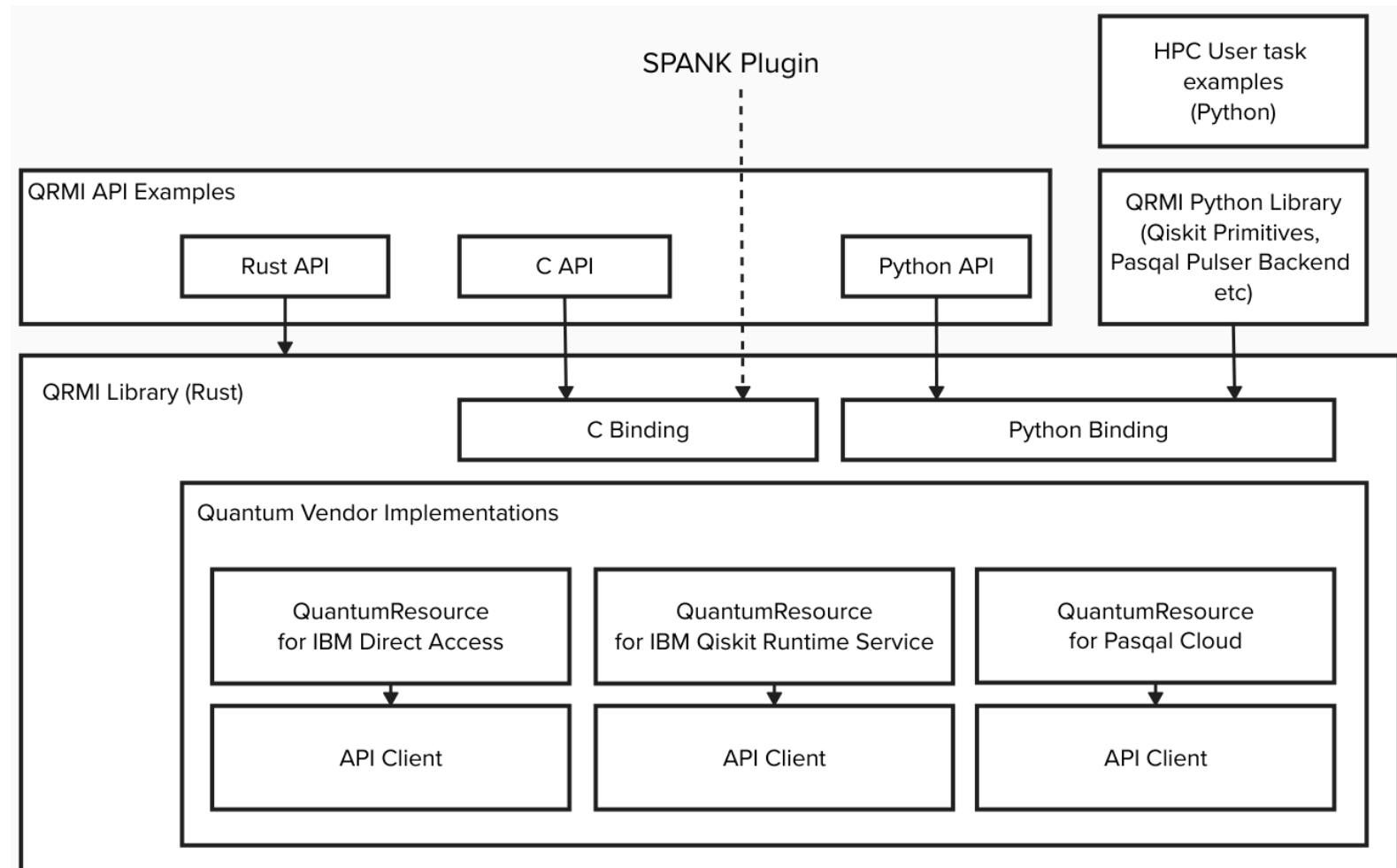
- Implement as Reverse Proxy, which is separated from Slurm components.
- QRMIs will submit request without Authorization header.
- Reverse Proxy can access secured credentials and get access token from IAM
- Reverse Proxy will forward the HTTP requests with adding Authorization and other headers

Auth Reverse Proxy



Source code structure

```
qrmi/  
├── dependencies/  
│   ├── direct_access_client/  
│   ├── pasqal_cloud_client/  
│   └── qiskit_runtime_client/  
├── examples/  
│   ├── pulser_backend/  
│   ├── qiskit_primitives/  
│   │   ├── ibm/  
│   │   └── pasqal/  
│   ├── qrmi/  
│   │   ├── c/  
│   │   ├── python/  
│   │   └── rust/  
│   └── task_runner/  
│       ├── pulser/  
│       ├── qiskit/  
│       └── qpp/  
├── python/  
│   └── qrmi/  
│       ├── primitives/  
│       │   ├── ibm/  
│       │   └── pasqal/  
│       └── pulser_backend/  
└── src/  
    ├── ibm/  
    └── pasqal/
```



Development Steps - Recommended

1. Your first step is to explore how you can enable HPC users to leverage your quantum resources effectively.

- as Qiskit Primitive ?
- or Custom APIs ?

2. Decide resource name

- Resource names we have today:
 - *“direct-access”*
 - *“qiskit-runtime-service”*
 - *“pasqal-cloud”*

3. Implement API Client to access your quantum backend

- Code commit to dependencies/<resource_name>

4. Implement QuantumResource

- Define Payload Data Format (input to Quantum resource)
 - *qrmi_payload_v1_schema.json*
 - *src/models/payload.rs*
- Implement *src/<vendor_name>/*
- Create QRMI API Examples and test your code

5. Implement user libraries to allow HPC users to use your QRMI

- Python ?
- Code commit to *python/qrmi/*
- Create examples and commit to *examples/*

From Integration to Innovation: QMRI and Slurm Driving Quantum Research

“The hardware experiments were managed via the quantum spank plugin for Slurm v0.1.0 [70].”



Jay Gambetta • Following
Director of IBM Research and IBM Fellow
[Visit my website](#)
3mo •

I am delighted to share a major milestone in our journey toward realizing useful quantum computing for chemical simulations.

The team introduced a novel, sample-based method for electronic structure calculations that is both scalable and backed by convergence guarantees. The algorithm, SqDRIFT, employs randomized unitary product formulas to enable quantum Krylov diagonalizations of molecular Hamiltonians.

In a 48-qubit experiment on `ibm_aachen` targeting the ground state of coronene, SqDRIFT achieved an accuracy surpassing that of classical post-Hartree-Fock methods. While Selected Configuration Interaction (SCI), the classical technique that inspired this family of sample-based approaches, still outperforms it, the gap is narrowing. These results, described in the preprint here: <https://lnkd.in/eGCSVE8w>, are highly promising for future quantum-centric supercomputing workflows.

This work was carried out in collaboration with [STFC Hartree Centre](#) through the HNCI partnership, leveraging the SLURM integration plugin (<https://lnkd.in/eRkVg7kM>) that we co-developed and released earlier this year. We were also honored to collaborate with Ali Alavi, director of the Max Planck Institute for Solid State Research and professor at the University of Cambridge, a pioneer in discrete QMC methods.

Quantum chemistry with provable convergence via randomized sample-based quantum diagonalization

Samuele Piccinelli,^{1,2,*} Alberto Baiardi,¹ Max Rossmannek,¹ Almudena Carrera Vazquez,¹ Francesco Tacchino,¹ Stefano Mensa,³ Edoardo Altamura,^{3,4} Ali Alavi,^{5,4} Mario Motta,⁶ Javier Robledo-Moreno,⁶ William Kirby,⁶ Kunal Sharma,⁶ Antonio Mezzacapo,⁶ and Ivano Tavernelli^{1,†}

¹IBM Quantum, IBM Research Europe - Zurich, CH-8803 Rüschlikon, Switzerland

²Institute of Physics, École Polytechnique Fédérale de Lausanne (EPFL), CH-1015 Lausanne, Switzerland

³The Hartree Centre, STFC, Sci-Tech Daresbury, Warrington, WA4 4AD, United Kingdom

⁴Yusuf Hamied Department of Chemistry, University of Cambridge, Lensfield Road, Cambridge CB2 1EW, United Kingdom

⁵Max Planck Institute for Solid State Research, Heisenbergstr. 1, 70569 Stuttgart, Germany

⁶IBM Quantum, IBM T.J. Watson Research Center, Yorktown Heights, NY 10598, United States

(Dated: August 5, 2025)

Sample-based quantum diagonalization (SQD) is a recently proposed algorithm to approximate the ground-state wave function of many-body quantum systems on near-term and early-fault-tolerant quantum devices. In SQD, the quantum computer acts as a sampling engine that generates the subspace in which the Hamiltonian is classically diagonalized. A recently proposed SQD variant, Sample-based Krylov Quantum Diagonalization (SKQD), uses quantum Krylov states as circuits from which samples are collected. Convergence guarantees can be derived for SKQD under similar assumptions to those of quantum phase estimation, provided that the ground-state wave function is concentrated, i.e., has support on a small subset of the full Hilbert space. Implementations of SKQD on current utility-scale quantum computers are limited by the depth of time-evolution circuits needed to generate Krylov vectors. For many complex many-body Hamiltonians of interest, such as the molecular electronic-structure Hamiltonian, this depth exceeds the capability of state-of-the-art quantum processors. In this work, we introduce a new SQD variant that combines SKQD with the qDRIFT randomized compilation of the Hamiltonian propagator. The resulting algorithm, termed SqDRIFT, enables SQD calculations at the utility scale on chemical Hamiltonians while preserving the convergence guarantees of SKQD. We apply SqDRIFT to calculate the electronic ground-state energy of several polycyclic aromatic hydrocarbons, up to system sizes beyond the reach of exact diagonalization.

I. INTRODUCTION

Quantum computing has emerged as a novel computational paradigm capable of addressing problems that exhibit unfavorable scaling on classical computers. Quantum chemistry is a prime candidate among these problems. In fact, quantum computers are believed to be able to speed up the solution of the electronic Schrödinger equation within a given basis set, which is a central computational challenge in quantum chemistry [1–3]. Robust quantum algorithms for solving the Schrödinger equation with performance guarantees already exist, and are mostly based on Quantum Phase Estimation (QPE) [4–

of VQE faces scalability challenges due to a steep measurement overhead [13] and the difficulty of navigating the optimization landscape [14]. Alternative strategies are required to scale-up quantum-chemical calculations on quantum computers beyond the reach of brute-force classical simulations – the so-called “quantum-utility” regime [15].

Methods exploiting the quantum computer to sample classically-hard probability distributions are emerging as a promising route towards this goal. Examples of this class of methods for natural-science applications are quantum algorithms inspired by classical selected configuration interaction (SCI) [16–21], such as quan-

Xiv:2508.02578v1 [quant-ph] 4 Aug 2025

Slurm Integration

Next Steps (T.B.D)

Requirements from HPC Community

- Accounting Information Support - QPU Usage & Resource utilization
- Network Middleware for security
- QRMI vendor library as Plugin
- Enhancing config file format for ease of defining multiple resources
- Accelerating Slurm integrations:
 - “--constraint” option
 - Quantum resource as GRES
- Improve error messages for self-service

Adoptions to upcoming IBM Quantum Software Enhancement

- Upcoming Qiskit releases
- Execution lane management by Direct Access

Contributing to HPC Community

- Enable new device support in QRMI
- Deploy to new data centers
- Documentation, Tutorials, Practice Guide etc.

Open Access to All Our Implementations

QRMI

Thin and vendor agnostic layer to access, control and monitor underlying on-premise or cloud quantum computers



SPANK Plugins

Slurm plugins for Quantum resources and jobs support

Also contains architecture documents (docs/)



